

NoSQL 数据库技术探究

李盛鹏

学号: 2351136

指导教师: 喻剑

摘要—本报告对 NoSQL 数据库技术进行系统性探究, 重点分析 Key-Value 存储 (Dynamo、Memcached)、列族存储 (Bigtable、Cassandra) 以及 NewSQL (CockroachDB) 等主流 NoSQL 系统的架构设计、技术演进与权衡决策。通过对比分析这些系统的设计哲学与关键技术点, 揭示了 NoSQL 数据库在一致性、可用性、扩展性之间的权衡规律, 以及从专用系统向通用系统演进的趋势。研究表明, NoSQL 数据库的发展经历了萌芽期、爆发期、成熟期和融合期四个阶段, 未来将向多模型融合、云原生和边缘计算方向演进。本报告基于 SOSP、OSDI、NSDI、VLDB、SIGMOD 等顶级会议论文的系统性分析, 结合工业实践案例, 为 NoSQL 数据库技术的理解与选型提供参考。

Index Terms—NoSQL 数据库, 分布式存储, CAP 定理, Bigtable, Dynamo, Cassandra, CockroachDB

I. 引言

A. 背景与意义

随着互联网技术的飞速发展, 数据规模呈现爆炸式增长。根据 IDC 预测, 到 2025 年全球数据总量将达到 175ZB。传统关系型数据库在海量数据场景下暴露出严重的扩展性瓶颈。Google、Amazon、Facebook 等互联网巨头率先面临这一挑战, 它们需要处理 PB 级甚至 EB 级的数据, 同时要求系统具备高可用性、高扩展性和低延迟等特性。

传统关系型数据库基于 ACID (原子性、一致性、隔离性、持久性) 事务模型, 虽然提供了强一致性保证, 但在分布式环境下面临以下困境:

- 扩展性瓶颈:** 单机架构难以水平扩展, 垂直扩展成本高昂。传统关系型数据库通过增加硬件配置 (CPU、内存、存储) 来提升性能, 但这种垂直扩展方式存在物理极限, 且成本呈指数增长。
- 性能瓶颈:** 严格的事务语义导致并发性能受限。ACID 事务要求所有操作要么全部成功, 要么全

部失败, 这种强一致性保证在高并发场景下会显著降低系统吞吐量。

- 可用性挑战:** 主从复制模式下, 主节点故障会导致服务中断。传统主从架构中, 写操作必须经过主节点, 一旦主节点故障, 整个系统将无法提供写服务。

为解决这些问题, NoSQL (Not Only SQL) 数据库应运而生。NoSQL 数据库放弃了部分 ACID 特性, 转而追求高可用性、高扩展性和高性能, 成为大数据时代的重要技术支撑。Google 的 Bigtable (2006 年) [1]、Amazon 的 Dynamo (2007 年) [2]、Facebook 的 Cassandra (2010 年) [3] 等系统的出现, 标志着 NoSQL 数据库技术的成熟与发展。

NoSQL 数据库的核心设计理念可以概括为: 在 CAP 定理的约束下, 根据应用场景的特点, 在一致性、可用性和分区容错性之间做出合适的权衡。这种权衡思想深刻影响了分布式系统的设计范式, 也为后续 NewSQL 数据库的发展奠定了基础。

B. 报告范围与研究方法

研究范围: 本报告聚焦以下主流 NoSQL 系统:

- Key-Value 存储:** Dynamo (Amazon)、Memcached (Facebook)
- 列族存储:** Bigtable (Google)、Cassandra (Facebook)
- 存储引擎:** RocksDB (Facebook)、MemC3 (CMU)
- NewSQL:** CockroachDB (Cockroach Labs)
- 多模型数据库:** ArangoDB、OrientDB

研究方法: 本报告基于顶会论文 (SOSP、OSDI、NSDI、VLDB、SIGMOD、CIDR 等) 的系统性分析与对比, 结合工业实践案例, 深入探讨 NoSQL 数据库的技术演进、设计权衡与未来趋势。

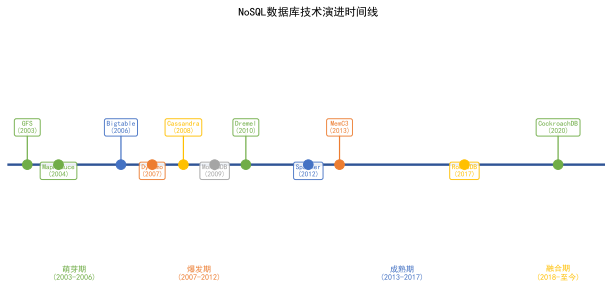


图 1: NoSQL 数据库技术演进时间线

数据来源: Google、Amazon、Facebook 等公司的工业实践与学术研究, 包括 Bigtable、Dynamo、Cassandra 等经典论文, 以及近年来的综述与对比研究。

本报告的组织结构如下: 第 2 节梳理 NoSQL 数据库的技术发展脉络; 第 3 节对核心系统进行对比分析; 第 4 节讨论技术权衡问题; 第 5 节展望未来趋势; 第 6 节总结全文。

II. 技术发展脉络

A. 时间线梳理

NoSQL 数据库的发展经历了四个主要阶段, 如图 1 所示。

萌芽期 (2003-2006): Google 内部系统探索

这一阶段以 Google 内部系统的探索为主要特征。2003 年, Google 发表了 GFS (Google File System) 论文 [12], 提出了分布式文件系统的设计范式, 为后续 NoSQL 系统提供了底层存储基础。2004 年, MapReduce 论文 [13] 发表, 提出了大规模数据处理的编程模型。2006 年, Bigtable 论文 [1] 在 OSDI 发表, 开创了列族存储的先河, 展示了如何在分布式环境下高效存储和管理结构化数据。

这些系统的设计思想深刻影响了后续 NoSQL 数据库的发展。GFS 的 Master-ChunkServer 架构、MapReduce 的分而治之思想、Bigtable 的稀疏多维映射模型, 都成为 NoSQL 数据库的经典设计范式。

爆发期 (2007-2012): 开源系统涌现

2007 年, Amazon 发表了 Dynamo 论文 [2], 这是一个具有里程碑意义的系统。Dynamo 首次系统性地展示了一致性哈希、向量时钟、Gossip 协议等技术在实际生产系统中的应用, 为高可用性 Key-Value 存储系统的设计提供了范式。

2008 年, Facebook 开源了 Cassandra 系统, 这是一个结合了 Dynamo 高可用性和 Bigtable 数据模型的

列族存储系统。Cassandra 的设计理念是“为互联网规模的应用而生”, 它继承了 Dynamo 的去中心化架构和 Bigtable 的列族数据模型, 成为 Apache 基金会的顶级项目。

2009 年, MongoDB 1.0 发布, 标志着文档数据库的兴起。MongoDB 采用 JSON-like 的文档模型, 提供了灵活的 Schema 设计, 深受开发者喜爱。

这一阶段的特征是: 学术研究开始向工业实践转化, 开源社区蓬勃发展, NoSQL 数据库从实验室走向生产环境。

成熟期 (2013-2017): 优化与融合

2013 年, MemC3 [5] 和 Scaling Memcache [6] 论文在 NSDI 发表, 展示了内存缓存技术的优化方向。MemC3 通过乐观 Cuckoo 哈希和 CLOCK 算法, 显著提升了 Memcached 的内存效率和吞吐量。

2017 年, RocksDB 论文 [7] 在 CIDR 发表, 这是一个基于 LSM-Tree 的嵌入式键值存储引擎。RocksDB 通过优化空间放大、读放大和写放大的权衡, 成为许多 NoSQL 系统的底层存储引擎。同年, PebblesDB [11] 在 SOSP 发表, 提出了基于 Fragmented Log-Structured Merge-Tree 的键值存储设计。

这一阶段的特征是: 系统优化成为研究重点, 存储引擎技术日趋成熟, NoSQL 系统开始向 NewSQL 演进。

融合期 (2018-至今): NoSQL 与 SQL 融合

2012 年, Spanner 论文 [10] 发表, 这是一个全球分布式数据库系统, 首次展示了如何在分布式环境下实现强一致性事务。2020 年, CockroachDB 论文 [8] 在 SIGMOD 发表, 这是一个开源的 NewSQL 数据库, 融合了 NoSQL 的扩展性与 SQL 的易用性。

2019 年, Lu 和 Holubova 发表了多模型数据库综述 [9], 系统性地分析了多模型数据库的发展趋势。多模型数据库试图在一个系统中支持文档、图、键值等多种数据模型, 代表了 NoSQL 与 SQL 融合的新方向。

这一阶段的特征是: NoSQL 与 SQL 的界限逐渐模糊, 系统设计追求在扩展性、一致性和易用性之间取得平衡。

B. 驱动因素分析

NoSQL 数据库的发展受到多重因素驱动, 这些因素相互作用, 共同塑造了技术演进的方向。表 I 总结了各系统面临的主要问题和驱动力。

表 I: NoSQL 系统面临的问题与驱动因素

系统	面临的问题	解决方案
Bigtable	单机无法存储 PB 级网页索引	GFS+Tablet 分片, 水平扩展
Dynamo	购物车服务必须 7×24 可用	去中心化架构, AP 优先
Cassandra	每天数十亿次 Inbox 写入	LSM-Tree+ 多级复制
MemC3	多核 CPU 下全局锁瓶颈	Cuckoo 哈希 + 细粒度锁
CockroachDB	全球部署的合规与一致性	Raft+HLC+ 地理分区

数据规模驱动

互联网应用的数据规模呈指数级增长, 对存储系统提出了前所未有的挑战。Google 的网页索引系统需要处理数十亿网页, 数据规模达到 PB 级别。Bigtable 论文 [1] 指出, Google 内部超过 60 个产品和项目使用 Bigtable, 包括 Google Analytics、Google Finance、Orkut 等, 这些应用的数据规模从 GB 到 PB 不等。

Amazon 的购物车系统需要支持数百万并发用户, 对高可用性有极高要求。Dynamo 论文 [2] 描述了 Amazon 平台的架构: 一个页面请求需要调用超过 150 个服务, 整个调用图可能有多层依赖。在这种环境下, 任何服务的不可用都会影响用户体验。

Facebook 的社交网络需要服务数亿用户, 每天产生数十亿写入操作。Cassandra 论文 [3] 指出, Inbox Search 功能需要处理每天数十亿次写入, 同时支持数亿用户的数据查询。

硬件发展驱动

硬件技术的发展为 NoSQL 数据库的演进提供了新的可能性。SSD 的普及显著提升了随机写入性能, 使得 LSM-Tree 架构的优势更加凸显。RocksDB 论文 [7] 指出, Facebook 的数据中心越来越多地使用 SSD 存储数据, 这促使 RocksDB 针对 SSD 特性进行优化。表 II 展示了硬件发展对 NoSQL 系统的影响。

多核 CPU 的普及带来了并发处理的需求, 锁优化成为关键。MemC3 论文 [5] 指出, 标准 Memcached 使用链表实现 LRU, 依赖全局锁保证一致性, 导致多核 CPU 下的性能瓶颈。MemC3 通过乐观 Cuckoo 哈希和细粒度锁, 显著提升了并发性能。

万兆网络的普及提升了数据中心内部的通信带宽, 使得分布式系统的设计更加灵活。CockroachDB 论文

表 II: 硬件发展对 NoSQL 系统的影响

硬件趋势	影响	受益系统
SSD 普及	随机写入性能提升	RocksDB, Cassandra
多核 CPU	并发处理需求	MemC3, Cassandra
万兆网络	跨地域通信带宽提升	CockroachDB, Spanner
大容量内存	内存缓存成为可能	Memcached, Redis

表 III: Bigtable 数据模型示例

行键	列键	时间戳	值
com.cnn.www	contents:	t6	"CNN..."
com.cnn.www	anchor:cnn.com	t9	"CNN"
com.cnn.www	anchor:my.look	t8	"CNN.com"

[8] 指出, 现代数据中心的网络延迟已经降低到亚毫秒级, 这使得跨地域的分布式事务成为可能。

业务需求驱动

互联网应用的业务需求对 NoSQL 数据库的设计产生了直接影响。7×24 小时可用性是电商、社交等场景的基本要求。Dynamo 论文 [2] 指出, Amazon 的购物车服务必须始终可用, 即使磁盘故障、网络波动或数据中心被龙卷风摧毁, 用户都应该能够查看和添加商品到购物车。

低延迟响应是用户体验的关键。MemC3 论文 [5] 指出, 互联网服务对数据访问延迟的要求越来越严格, 许多系统选择将全部或大部分数据集保存在内存中, 以提供毫秒级的访问延迟。

全球化部署带来了跨地域数据同步与合规要求。CockroachDB 论文 [8] 指出, 欧盟的 GDPR 法规要求个人数据必须存储在欧盟境内, 这促使数据库系统支持地理分区和数据驻留策略。

III. 核心系统对比分析

A. Bigtable (Google, 2006)

Bigtable 是一个稀疏的、分布式的、持久化的多维有序映射, 数据模型为 (行键, 列键, 时间戳) → 字符串。其架构如图 2 所示。

Bigtable 的架构包括以下核心组件:

- 1) **Master Server:** 负责 Tablet 分配与负载均衡。Master Server 维护 Tablet 的位置信息, 处理 Tablet Server 的加入和退出, 以及 Tablet 的分裂和合并。

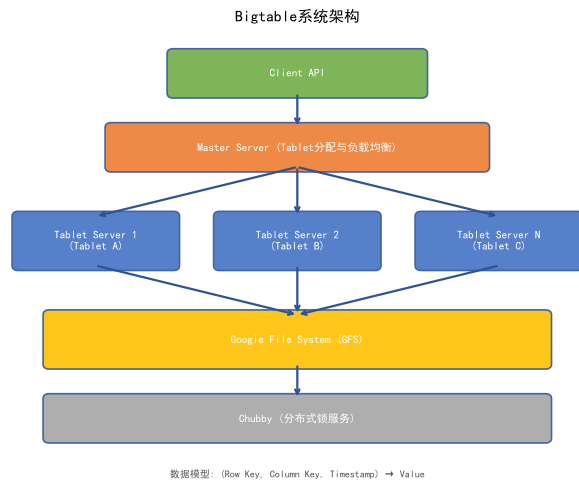


图 2: Bigtable 系统架构

- 2) **Tablet Server:** 存储和管理 Tablet (数据分片)。每个 Tablet Server 负责一组 Tablet，处理客户端的读写请求。
- 3) **GFS:** 底层分布式文件系统，提供持久化存储。Tablet 的数据和日志都存储在 GFS 中。
- 4) **Chubby:** 分布式锁服务，用于协调和元数据管理。Chubby 确保只有一个 Master Server 处于活动状态，并存储 Tablet 的位置信息。

关键设计决策

行键设计: Bigtable 采用反转 URL 主机名作为行键 (如“com.cnn.www”), 实现同一域名数据的局部性。这种设计使得同一网站的网页数据在物理存储上相邻, 大大提高了范围查询的效率。Bigtable 论文 [1] 指出, 在 Webtable 中, 将同一域名的页面存储在连续的行中, 使得主机和域名分析更加高效。

列族机制: 列键被分组为列族 (Column Family), 形成访问控制和压缩的基本单位。列族必须预先创建, 但列可以动态添加。这种设计既提供了灵活性, 又保持了良好的数据组织结构。Bigtable 论文 [1] 指出, 列族的数量应该很少 (最多几百个), 且很少变化, 而列的数量可以是无界的。

Tablet 分裂: 当 Tablet 大小超过阈值时, Bigtable 会自动将其分裂为多个子 Tablet, 实现动态负载均衡。这种机制使得系统能够自动适应数据增长, 无需人工干预。

适用场景

Bigtable 适用于大规模结构化数据存储 (网页索

表 IV: Bigtable 典型应用案例

应用	数据规模	特点
Google Analytics	PB 级	高写入吞吐
Google Finance	TB 级	低延迟读取
Orkut	PB 级	社交图谱
Google Earth	PB 级	地理数据

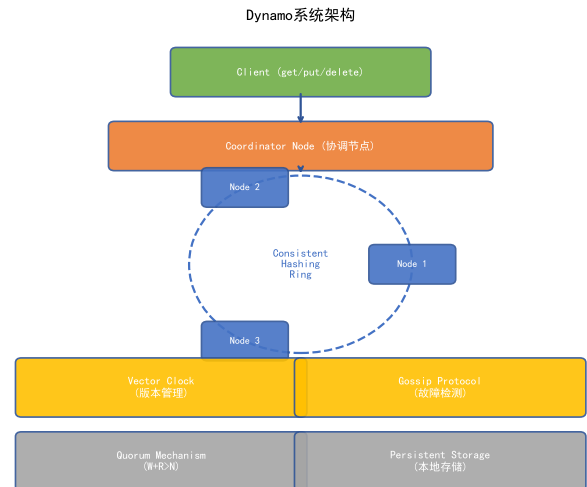


图 3: Dynamo 系统架构

引、用户行为日志)、批量分析与实时服务混合场景、需要灵活 Schema 的应用。Bigtable 论文 [1] 指出, Bigtable 被用于 Google Analytics、Google Finance、Orkut、Personalized Search、Writely 和 Google Earth 等 60 多个产品和项目。表 IV总结了 Bigtable 的典型应用案例。

B. Dynamo (Amazon, 2007)

Dynamo 是一个高度可用的 Key-Value 存储系统, 采用去中心化架构。其架构如图 3所示。

Dynamo 的核心组件包括:

- 1) **一致性哈希环:** 数据分区与负载均衡。每个节点被分配一个随机位置, 数据项通过哈希函数映射到环上的一个位置, 然后顺时针找到第一个节点存储。
- 2) **向量时钟:** 版本管理与冲突检测。每个写操作都会生成一个新的向量时钟, 用于跟踪数据版本和检测并发更新冲突。
- 3) **Gossip 协议:** 去中心化的故障检测与成员管理。每个节点定期与其他节点交换信息, 最终所有节

表 V: Dynamo 不同场景的 Quorum 配置

场景	W	R	一致性
购物车	1	1	最终一致
会话管理	1	1	最终一致
产品目录	2	2	强一致

点都能感知整个集群的状态。

- 4) **Quorum 机制**: 可配置的读写一致性级别。通过调整 W（写入确认数）和 R（读取确认数）的值，可以在一致性和可用性之间进行权衡。

关键设计决策

一致性哈希: Dynamo 使用一致性哈希将数据分布到多个节点上。每个数据项通过哈希函数映射到哈希环上的一个位置，然后顺时针找到第一个节点存储。节点增减只影响相邻节点，数据迁移成本低。Dynamo 论文 [2] 指出，一致性哈希的主要优势是：节点的加入或离开只影响其直接邻居，其他节点保持不变。

向量时钟: Dynamo 使用向量时钟（Vector Clock）来跟踪数据版本。每个写操作都会生成一个新的向量时钟，用于检测并发更新冲突。当检测到冲突时，系统将冲突版本返回给客户端，由应用层解决。Dynamo 论文 [2] 指出，向量时钟使得 Dynamo 能够在没有中央协调的情况下检测并发更新。

Quorum 机制: Dynamo 支持可配置的读写一致性级别（ $W+R>N$ ）。例如，设置 $W=2$ 、 $R=2$ 、 $N=3$ 时，写入需要 2 个节点确认，读取需要从 2 个节点获取数据，确保读写一致性。Dynamo 论文 [2] 指出，通过调整 W 和 R 的值，开发者可以根据应用需求在一致性、可用性和性能之间进行权衡。

适用场景

Dynamo 适用于购物车、用户偏好等高可用性场景，可接受最终一致性的应用，需要低延迟写入的场景。Dynamo 论文 [2] 指出，Dynamo 被用于 Amazon 的购物车服务、会话管理、产品目录等核心服务。表 V 总结了 Dynamo 在不同场景下的配置选择。

C. Cassandra (Facebook, 2010)

Cassandra 结合了 Dynamo 的高可用性和 Bigtable 的数据模型，采用去中心化架构。其架构如图 4 所示。

Cassandra 的核心组件包括：

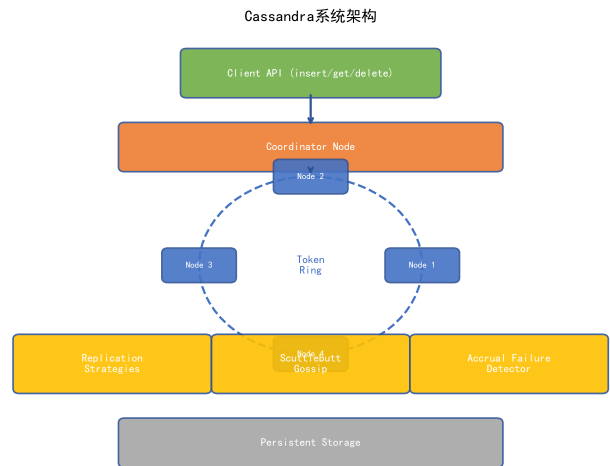


图 4: Cassandra 系统架构

- 1) **Token Ring**: 一致性哈希环，负责数据分区。每个节点被分配一个 Token，数据项根据 Token 值存储到对应的节点。
- 2) **Replication Strategies**: 多级复制策略（Rack Unaware、Rack Aware、Datacenter Aware）。应用可以根据需求选择合适的复制策略。
- 3) **Scuttlebutt Gossip**: 高效的成员管理协议。Cassandra 使用 Scuttlebutt 实现去中心化的成员管理和故障检测。
- 4) **Accrual Failure Detector**: 累积故障检测器。与传统的布尔型故障检测器不同，累积故障检测器输出怀疑级别，能够更好地适应网络波动。

关键设计决策

保序哈希: Cassandra 使用保序哈希函数，保持数据的排序顺序，支持范围查询。同时，通过虚拟节点（VNode）实现负载均衡，避免数据倾斜。Cassandra 论文 [3] 指出，保序哈希使得 Cassandra 能够支持范围查询，这是 Dynamo 不支持的功能。

多级复制策略: Cassandra 支持三种复制策略：

- **Rack Unaware**: 不考虑机架位置，随机选择副本
- **Rack Aware**: 在同一数据中心内，确保副本分布在不同机架
- **Datacenter Aware**: 跨数据中心复制，提高容灾能力

Cassandra 论文 [3] 指出，Datacenter Aware 策略使得 Cassandra 能够处理整个数据中心故障，这是 Facebook 部署的关键需求。

表 VI: Cassandra 复制策略对比

策略	特点	适用场景
Rack Unaware	简单, 不考虑拓扑	单数据中心
Rack Aware	机架感知	中等规模部署
Datacenter Aware	跨数据中心	全球化部署

表 VII: Memcached 与 MemC3 性能对比

指标	Memcached	MemC3
元数据开销	18n 字节	9.7n 字节
内存效率	基准	提升 30%
并发锁	全局锁	细粒度锁
吞吐量	基准	提升 3 倍

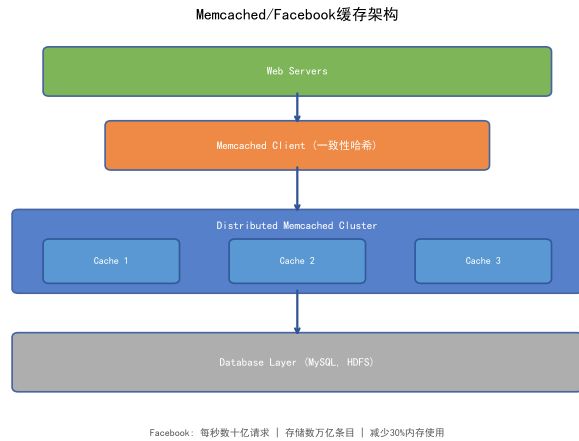


图 5: Memcached/Facebook 缓存架构

累积故障检测器: Cassandra 使用累积故障检测器 (Accrual Failure Detector), 输出怀疑级别而非布尔值。这种设计能够更好地适应网络波动, 减少误判。Cassandra 论文 [3] 指出, 累积故障检测器的输出是一个表示怀疑程度的值, 而不是简单的”上”或”下”。

适用场景

Cassandra 适用于社交网络、物联网等高写入场景, 需要跨数据中心复制的应用, 需要灵活 Schema 和高可用性的系统。Cassandra 论文 [3] 指出, Cassandra 被用于 Facebook 的 Inbox Search 功能, 该功能需要处理每天数十亿次写入。表 VI 总结了 Cassandra 的复制策略对比。

D. Memcached/Facebook 实践 (2013)

Memcached 是一个简单的内存缓存系统, 提供 set/get/delete 操作。Facebook 在其基础上构建了大规模分布式缓存集群。其架构如图 5 所示。

Facebook 的 Memcached 架构包括:

- 1) **Web Servers:** 接收用户请求, 构建页面内容。

- 2) **Memcached Client:** 一致性哈希路由到缓存节点。客户端负责将请求路由到正确的缓存服务器。
- 3) **Distributed Memcached Cluster:** 多个缓存节点组成集群, 提供高吞吐量和低延迟的缓存服务。
- 4) **Database Layer:** 后端数据库 (MySQL、HDFS 等), 提供持久化存储。

关键设计决策

一致性哈希: Memcached 使用一致性哈希将数据分布到多个缓存节点上, 实现负载均衡和水平扩展。Scaling Memcache 论文 [6] 指出, 一致性哈希使得 Memcached 集群能够动态扩展, 无需重新分配所有数据。

多集群部署: Facebook 部署了多个地理分布的 Memcached 集群, 通过跨地域复制提高可用性。当某个集群故障时, 请求可以路由到其他集群。Scaling Memcache 论文 [6] 指出, Facebook 的 Memcached 部署跨越多个数据中心, 每秒处理数十亿请求。

MemC3 优化: MemC3 (Memcached with CLOCK and Concurrent Cuckoo Hashing) 通过乐观 Cuckoo 哈希和 CLOCK 算法, 减少 30% 内存使用, 吞吐量提升 3 倍。MemC3 论文 [5] 指出, 传统 Memcached 使用链表实现 LRU, 需要 18n 字节的元数据, 而 MemC3 只需要 9.7n 字节。

表 VII 对比了传统 Memcached 与 MemC3 的性能差异。

适用场景

Memcached 适用于高并发读取的 Web 应用、数据库查询结果缓存、会话存储和临时数据缓存。Scaling Memcache 论文 [6] 指出, Facebook 的 Memcached 部署存储了数万亿条目, 每秒处理超过十亿请求。表 VIII 展示了 Facebook Memcached 部署的关键指标。

表 VIII: Facebook Memcached 部署关键指标

指标	数值
条目数量	数万亿 (trillions)
请求速率	每秒数十亿 (billions/sec)
集群规模	多个数据中心
内存节省	MemC3 减少 30%

CockroachDB 分层架构

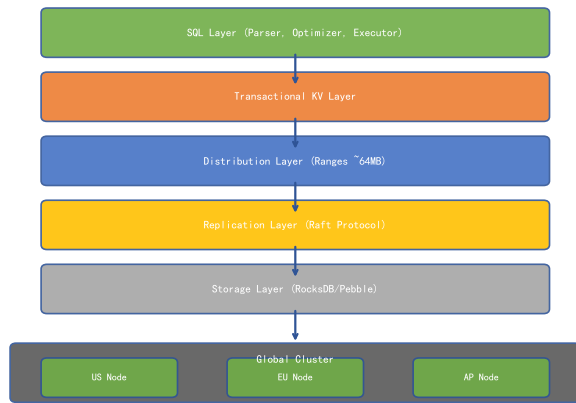


图 6: CockroachDB 分层架构

E. CockroachDB (2020)

CockroachDB 是一个分布式 SQL 数据库，融合了 NoSQL 的扩展性与 SQL 的易用性。其架构如图 6 所示。

CockroachDB 的分层架构包括：

- 1) **SQL Layer:** 解析器、优化器、执行器。SQL 层将高级 SQL 语句转换为底层的键值操作。
- 2) **Transactional KV Layer:** 原子性和隔离性保证。这一层负责确保跨多个键值对的变更的原子性。
- 3) **Distribution Layer:** 数据分区与路由。这一层将逻辑键空间划分为多个 Range（约 64MB），并在集群中分布。
- 4) **Replication Layer:** 基于 Raft 的复制协议。每个 Range 维护多个副本，通过 Raft 日志同步状态。
- 5) **Storage Layer:** RocksDB/Pebble 存储引擎。底层存储使用 RocksDB 或 Pebble，提供持久化存储。

关键设计决策

地理分区：CockroachDB 支持地理分区，用户可以

表 IX: CockroachDB 核心特性

特性	说明
地理分区	数据可存储在指定地理位置
Raft 复制	强一致性复制协议
HLC 时钟	混合逻辑时钟实现全局一致性
PostgreSQL 兼容	支持 PostgreSQL 协议和 SQL 方言
云中立	跨公有云和私有云部署

指定数据的存储位置，满足 GDPR 等合规要求。例如，欧洲用户的数据可以存储在欧洲数据中心。CockroachDB 论文 [8] 指出，地理分区使得组织能够控制数据的物理位置，满足性能和合规需求。

Raft 复制：CockroachDB 使用 Raft 共识协议实现强一致性复制。每个分区维护多个副本，通过 Raft 日志同步状态，确保数据一致性。CockroachDB 论文 [8] 指出，Raft 协议提供了强一致性保证，同时支持自动故障恢复。

混合逻辑时钟：CockroachDB 使用混合逻辑时钟（HLC）实现全局一致性，无需原子钟（如 Spanner 的 TrueTime），只需 NTP 同步即可。CockroachDB 论文 [8] 指出，HLC 结合了物理时钟和逻辑时钟的优点，能够在松散同步的时钟环境下实现强一致性。

适用场景

CockroachDB 适用于全球化部署的 OLTP 应用、需要强一致性的金融、电商系统、需要 SQL 接口和事务支持的场景。CockroachDB 论文 [8] 指出，CockroachDB 被用于需要跨地域部署、强一致性和 SQL 接口的企业级应用。表 IX 总结了 CockroachDB 的核心特性。

F. 系统综合对比

为了更全面地理解各 NoSQL 系统的特点，表 X 从多个维度对本报告分析的系统进行了综合对比。

IV. 技术权衡讨论

A. 一致性 vs 可用性 vs 分区容错（CAP 定理）

CAP 定理指出，分布式系统无法同时满足一致性（C）、可用性（A）、分区容错性（P）三者。各系统在设计时做出了不同的权衡，如图 7 所示。

Bigtable (CP): Bigtable 依赖 GFS 和 Chubby，提供强一致性。当网络分区发生时，系统可能暂时不可用，但保证数据一致性。Bigtable 论文 [1] 指出，Bigtable 的设计目标是提供强一致性，而不是高可用性。

表 X: NoSQL 系统综合对比

维度	Bigtable	Dynamo	Cassandra	Memcached	CockroachDB
发布年份	2006	2007	2010	2003/2013	2020
发布机构	Google	Amazon	Facebook	Facebook	Cockroach Labs
数据模型	列族	Key-Value	列族	Key-Value	关系型
CAP 选择	CP	AP	AP/CP 可配置	AP	CP
一致性模型	强一致	最终一致	可配置	最终一致	强一致
复制协议	GFS 复制	无	可调	无	Raft
存储引擎	GFS	本地存储	LSM-Tree	内存	RocksDB/Pebble
扩展方式	水平扩展	水平扩展	水平扩展	水平扩展	水平扩展
典型场景	分析 +OLTP	高可用 OLTP	高写入 OLTP	缓存	全球 OLTP
开源状态	HBase 实现	已开源	Apache 项目	已开源	已开源

各系统CAP权衡对比

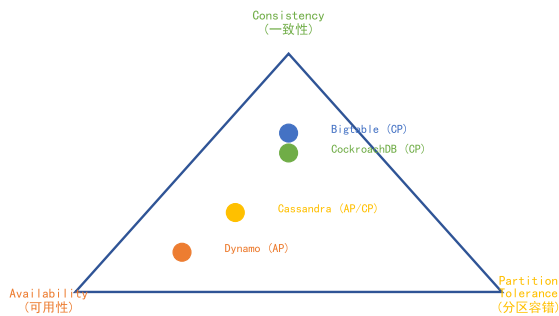


图 7: 各系统 CAP 权衡对比

Dynamo (AP): Dynamo 选择高可用性，允许最终一致性。在网络分区期间，系统仍可读写，通过版本向量检测冲突。Dynamo 论文 [2] 指出，Amazon 的购物车服务需要高可用性，即使牺牲一致性也在所不惜。

Cassandra (AP/CP 可配置): Cassandra 默认 AP，但可通过调整一致性级别（Quorum）实现 CP 行为。Cassandra 论文 [3] 指出，Cassandra 的设计允许开发者根据应用需求在一致性和可用性之间进行权衡。

CockroachDB (CP): CockroachDB 使用 Raft 协议，提供强一致性（可序列化隔离），在网络分区时牺牲部分可用性。CockroachDB 论文 [8] 指出，CockroachDB 的设计目标是提供强一致性，同时保持高可用性。

表 XI总结了各系统的 CAP 权衡选择。

B. PACELC 定理：CAP 的扩展

CAP 定理虽然经典，但它只考虑了网络分区这一种故障场景。在实际系统中，即使网络正常，系统也

表 XI: 各系统 CAP 权衡选择

系统	一致性	可用性	分区容错
Bigtable	强	中	是
Dynamo	最终	高	是
Cassandra	可配置	高	是
Memcached	最终	高	是
CockroachDB	强	高	是

需要在延迟和一致性之间做出权衡。Daniel Abadi 在 2012 年提出的 PACELC 定理 [14] 扩展了 CAP 定理，更全面地描述了分布式系统的设计权衡。

PACELC 定理指出：如果发生分区（Partition），系统必须在可用性（A）和一致性（C）之间选择；否则（Else），系统必须在延迟（Latency）和一致性（Consistency）之间选择。用公式表示为：

$$\text{PACELC} = \begin{cases} (A, L) & \text{Dynamo} \\ (C, L) & \text{Bigtable} \\ (A, C) & \text{Cassandra (默认)} \\ (C, C) & \text{CockroachDB} \end{cases} \quad (1)$$

表 XII总结了各系统的 PACELC 权衡选择。

PACELC 定理揭示了一个重要洞察：大多数 NoSQL 系统在正常运行时也需要在延迟和一致性之间做出权衡。例如，Dynamo 在正常运行时选择低延迟（最终一致性），而 CockroachDB 则始终选择强一致性，即使这意味着更高的延迟。

C. 读优化 vs 写优化

读优化系统：

表 XII: 各系统 PACELC 权衡选择

系统	分区时	正常时
Bigtable	一致性	低延迟
Dynamo	可用性	低延迟
Cassandra	可用性	可配置
Memcached	可用性	低延迟
CockroachDB	一致性	一致性

Bigtable 使用 Bloom Filter 和布局优化, 减少读放大。Bigtable 论文 [1] 指出, Bloom Filter 可以显著减少读操作需要访问的磁盘数量。

Memcached 提供内存缓存, 亚毫秒级读取。Scaling Memcache 论文 [6] 指出, Memcached 的读取延迟通常在亚毫秒级别。

Dremel 使用列式存储, 减少 I/O。Dremel 论文 [4] 指出, 列式存储可以显著减少读操作需要扫描的数据量。

写优化系统:

Dynamo 使用 LSM-Tree 结构, 顺序写入。LSM-Tree 将写操作缓冲到内存, 然后批量刷新到磁盘, 避免随机写入。

Cassandra 使用 LSM-Tree 和写合并, 实现高写入吞吐量。Cassandra 论文 [3] 指出, LSM-Tree 使得 Cassandra 能够处理每天数十亿次写入。

RocksDB 优化 LSM-Tree, 减少写放大。RocksDB 论文 [7] 指出, RocksDB 通过优化 LSM-Tree 的合并策略, 显著减少了写放大。

LSM-Tree (Log-Structured Merge-Tree) 是写优化存储引擎的核心技术。它将写操作缓冲到内存, 然后批量刷新到磁盘, 避免随机写入。但读取时需要多级查找, 可能产生读放大。

读写放大的权衡

LSM-Tree 存储引擎在读写性能之间存在固有的权衡。写放大 (Write Amplification) 是指实际写入磁盘的数据量与用户写入数据量的比值。读放大 (Read Amplification) 是指读取一个键值对需要访问的磁盘块数量。空间放大 (Space Amplification) 是指实际占用空间与用户数据大小的比值。

表 XIII总结了不同存储引擎在读写放大方面的权衡。

PebblesDB 论文 [11] 提出了一种新的权衡方式: 通过引入 Fragmented Log-Structured Merge-Tree,

表 XIII: 存储引擎读写放大权衡

存储引擎	写放大	读放大	空间放大
B-Tree	高	低 ($O(1)$)	低
LSM-Tree	中	中 ($O(L)$)	高
PebblesDB	低	中	中
RocksDB	优化	优化	优化

表 XIV: 各系统通用性与专用性对比

系统	类型
Bigtable	通用型
Dynamo	专用型
Cassandra	通用型
Memcached	专用型
CockroachDB	通用型

PebblesDB 在保持低写放大的同时, 显著降低了读放大。实验表明, PebblesDB 的写吞吐量比 LevelDB 高 6.7 倍, 比 RocksDB 高 4.5 倍。

D. 通用性 vs 专用性

通用型系统:

Bigtable 支持多种 Google 产品 (网页索引、金融、地球)。Bigtable 论文 [1] 指出, Bigtable 被用于 60 多个产品和项目, 展示了其通用性。

Cassandra 支持 Facebook 内部多个服务。Cassandra 论文 [3] 指出, Cassandra 被用于 Inbox Search、消息传递等多个服务。

CockroachDB 支持 SQL 标准, 兼容 PostgreSQL。CockroachDB 论文 [8] 指出, CockroachDB 的设计目标是提供一个通用的分布式 SQL 数据库。

专用型系统:

Dynamo 专为 Amazon 购物车设计。Dynamo 论文 [2] 指出, Dynamo 的设计目标是满足 Amazon 购物车服务的特定需求。

Memcached 专为缓存场景设计。Scaling Memcache 论文 [6] 指出, Memcached 的设计目标是提供高性能的内存缓存服务。

Dremel 专为交互式分析设计。Dremel 论文 [4] 指出, Dremel 的设计目标是支持大规模数据集的交互式查询。

表 XIV总结了各系统的通用性与专用性。

通用型系统的优势在于一套系统支持多种场景,降低运维成本;专用型系统的优势在于针对特定场景深度优化,性能更优。未来趋势是多模型数据库,试图兼顾通用性与专用性。

V. 未来趋势展望

A. 趋势一: 多模型融合

多模型数据库能够在一个系统中支持文档、图、键值等多种数据模型,提供统一查询接口。代表性系统包括 ArangoDB、OrientDB、MongoDB 等。这种趋势降低了系统复杂性,提高了开发效率。

Lu 和 Holubova 的综述 [9] 指出,多模型数据库的兴起是为了解决数据多样性带来的挑战。现代应用通常需要处理多种类型的数据:结构化数据(关系表)、半结构化数据(JSON、XML)、非结构化数据(文本、图像)和图数据(社交网络)。传统的做法是使用多个专门的数据库系统(多语言持久化),但这会带来系统复杂性和数据一致性问题。

多模型数据库的优势包括:

- 统一查询接口: 开发者可以使用单一 API 访问不同类型的数据
- 事务支持: 跨数据模型的事务保证数据一致性
- 降低运维成本: 只需要管理一个数据库系统
- 数据关联: 不同类型的数据可以自然关联,无需复杂的 JOIN 操作

代表性系统如 ArangoDB 支持三种数据模型: 文档(JSON)、图(节点和边)和键值。开发者可以使用 AQL (ArangoDB Query Language) 统一查询这些不同类型的数据。

B. 趋势二: 云原生数据库

云原生数据库具备弹性扩展、多云部署、Serverless 等特性。CockroachDB、Spanner 等系统支持跨云部署,避免供应商锁定。Serverless 数据库按使用量付费,无需管理基础设施。

CockroachDB 论文 [8] 指出, CockroachDB 的设计目标之一是“云中立”,即一个 CockroachDB 集群可以跨越任意数量的不同公有云和私有云。这种设计使得用户可以避免供应商锁定,例如依赖某个云提供商的专有 SQL 扩展,或暴露于云提供商的故障。

云原生数据库的关键特性包括:

- 弹性扩展: 根据负载自动扩缩容,无需人工干预

- 多云部署: 支持跨云部署,提高可用性和容灾能力
- Serverless: 按使用量付费,无需管理基础设施
- 自动故障恢复: 节点故障时自动恢复,无需人工干预

Spanner 论文 [10] 展示了全球分布式数据库的可能性,而 CockroachDB 则将这种能力开源,使得更多组织能够构建全球化部署的应用。

C. 趋势三: 边缘计算与分布式数据

随着 IoT 和 5G 的发展,边缘计算成为新趋势。数据处理从云端下沉到边缘节点,需要新的分布式数据管理技术。Cassandra、CockroachDB 等系统开始支持边缘部署场景。

边缘计算对数据库系统提出了新的挑战:

- 低延迟: 边缘节点需要毫秒级的数据访问延迟
- 离线能力: 边缘节点可能暂时与云端断开连接
- 数据同步: 边缘与云端的数据需要保持一致
- 资源受限: 边缘节点的计算和存储资源有限

Cassandra 的去中心化架构使其适合边缘部署场景。每个边缘节点可以运行一个 Cassandra 实例,独立处理本地数据,同时与其他节点保持数据同步。Cassandra 论文 [3] 指出, Cassandra 的架构设计使得它能够处理整个数据中心故障,这种能力同样适用于边缘节点故障的场景。

CockroachDB 的地理分区功能也使其适合边缘部署。CockroachDB 论文 [8] 指出,地理分区使得数据可以存储在离用户最近的位置,减少访问延迟。

VI. 结论

本报告对 NoSQL 数据库技术进行了系统性探究,分析了 Bigtable、Dynamo、Cassandra、Memcached、CockroachDB 等主流系统的架构设计、技术演进与权衡决策。主要结论包括:

- 1) NoSQL 数据库解决了海量数据存储的扩展性问题。通过放弃部分 ACID 特性, NoSQL 数据库实现了高可用性、高扩展性和高性能,成为大数据时代的重要技术支撑。Bigtable、Dynamo、Cassandra 等系统的成功实践证明了一点。
- 2) 各系统在 CAP 权衡中做出了不同的设计选择。Bigtable 和 CockroachDB 选择强一致性 (CP), Dynamo 选择高可用性 (AP), Cassandra 支持可

配置的一致性级别。这些选择反映了不同应用场景的需求差异。

- 3) **存储引擎优化显著提升了系统性能。**LSM-Tree、Bloom Filter、Cuckoo Hashing 等技术的应用,显著提升了 NoSQL 系统的读写性能和内存效率。RocksDB、MemC3 等系统展示了存储引擎优化的重要性。
- 4) **NoSQL 与 SQL 的融合是未来趋势。**CockroachDB 等 NewSQL 系统证明了在保持 SQL 接口和事务支持的同时,实现高扩展性是可能的。多模型数据库的兴起进一步模糊了 NoSQL 与 SQL 的界限。

NoSQL 数据库的发展历程体现了技术演进的规律:从专用到通用,从单模型到多模型,从单地域到全球。这些演进方向为数据库技术的未来发展提供了重要参考。

未来的研究方向包括:

- 多模型数据库的性能优化和查询优化
- 云原生数据库的弹性扩展和自动调优
- 边缘计算场景下的分布式数据管理
- AI 驱动的数据库自适应优化

参考文献

- [1] F. Chang, J. Dean, S. Ghemawat, et al., "Bigtable: A distributed storage system for structured data," in *Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2006, pp. 205-218.
- [2] G. DeCandia, D. Hastorun, M. Jampani, et al., "Dynamo: Amazon's highly available key-value store," in *Proc. ACM Symp. Operating Systems Principles (SOSP)*, 2007, pp. 205-220.
- [3] A. Lakshman and P. Malik, "Cassandra: A decentralized structured storage system," *ACM SIGOPS Operating Systems Review*, vol. 44, no. 2, pp. 35-40, 2010.
- [4] S. Melnik, A. Gubarev, J. J. Long, et al., "Dremel: Interactive analysis of web-scale datasets," *Proc. VLDB Endowment*, vol. 3, no. 1-2, pp. 330-339, 2010.
- [5] B. Fan, D. G. Andersen, and M. Kaminsky, "MemC3: Compact and concurrent memcache with dumber caching and smarter hashing," in *Proc. USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2013, pp. 371-384.
- [6] R. Nishtala, H. Fugal, S. Grimm, et al., "Scaling memcache at Facebook," in *Proc. USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2013, pp. 385-398.
- [7] S. Dong, M. Callaghan, L. Galanis, et al., "Optimizing space amplification in RocksDB," in *Proc. Biennial Conf. Innovative Data Systems Research (CIDR)*, 2017.
- [8] R. Taft, I. Sharif, A. Matei, et al., "CockroachDB: The resilient geo-distributed SQL database," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2020, pp. 1493-1506.
- [9] J. Lu and I. Holubova, "Multi-model databases: A new journey to handle the variety of data," *ACM Computing Surveys*, vol. 52, no. 3, pp. 1-38, 2019.
- [10] J. C. Corbett, J. Dean, M. Epstein, et al., "Spanner: Google's globally distributed database," *ACM Transactions on Computer Systems*, vol. 31, no. 3, pp. 1-22, 2013.
- [11] P. Raju, R. Kadekodi, K. Krish, et al., "PebblesDB: Building key-value stores using fragmented log-structured merge trees," in *Proc. ACM Symp. Operating Systems Principles (SOSP)*, 2017, pp. 589-604.
- [12] S. Ghemawat, H. Gobioff, and S.-T. Leung, "The Google File System," in *Proc. ACM Symp. Operating Systems Principles (SOSP)*, 2003, pp. 29-43.
- [13] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2004, pp. 137-150.
- [14] D. Abadi, "Consistency tradeoffs in modern distributed database system design," *IEEE Computer*, vol. 45, no. 2, pp. 37-42, 2012.
- [15] Google, "LevelDB: A fast key-value storage library written at Google," GitHub Repository, 2011. [Online]. Available: <https://github.com/google/leveldb>
- [16] Redis Labs, "Redis: In-memory data structure store," 2023. [Online]. Available: <https://redis.io>
- [17] MongoDB Inc., "MongoDB: The database for modern applications," 2023. [Online]. Available: <https://www.mongodb.com>
- [18] Apache Software Foundation, "Apache HBase: Apache Hadoop database," 2023. [Online]. Available: <https://hbase.apache.org>
- [19] CockroachDB, "Pebble: A fast key-value store designed for CockroachDB," GitHub Repository, 2023. [Online]. Available: <https://github.com/cockroachdb/pebble>